HRL Best Practices for Tierwise Exploration–Exploitation Control

Codex-generated Scholarship Request: HRL Best Practices for Tierwise Exploration–Exploitation Control

Provenance and framing note

This request was prepared from a live design session between the Project Owner (PO) and an engineering consultant. The PO clarified that the central novelty of the `state_collapser` project is *not* supposed to be a reinvention of generic hierarchical training control. Rather, the PO's core idea is that many RL problems admit useful *non-canonical* hierarchical structure even when they do not come with any obvious intrinsic decomposition into meaningful subtasks, and that this hidden structure can be induced by quotienting / contraction methods. In the course of the discussion, the PO further clarified that the open question is likely not whether a tier should become absolutely "closed," but whether each tier should instead carry its own evolving exploration–exploitation balance, possibly with mechanisms for re-initiating exploration when novelty or instability reappears. This document was created to request scholarship specifically on that latter control question, with the PO's intent being to separate the genuinely novel part of the project (hierarchy induction) from the parts that should, where possible, be inherited from existing HRL best practices.

Setup

I am developing a mathematical and software framework for hierarchical reinforcement learning called `state_collapser`. The novel part of the framework is *not* the generic HRL control logic itself. The novelty is instead a mechanism for inducing *non-canonical hierarchical structure* on RL problems that do not come with any obvious intrinsic decomposition into meaningful subtasks.

I need a literature-facing analysis of the following specific question.

Core question

Suppose an RL problem has been organized into a hierarchy of tiers. In my case, these tiers are not necessarily intrinsic subtasks; they may instead be quotient- or contraction-induced abstractions whose states need not have direct executable meaning, so long as they lift back to executable lower-level behavior.

What are the *best practices from the HRL literature* for controlling the balance between:

- exploration,
- exploitation / policy training,
- and possible re-initiation of exploration

at different hierarchical levels?

Important clarification

I am *not* asking for a new algorithm invented from scratch.

I am explicitly asking whether standard or near-standard HRL practice already contains reusable answers to the following control problem:

> At each tier of a hierarchy, should there simply be an exploration–exploitation tradeoff ratio, possibly changing over time, rather than some absolute notion of "closing" a tier once and for all?

In other words, I want to know whether the right framework is something like:

- each tier always remains available,
- each tier has its own exploration / exploitation balance,
- higher-level or lower-level exploration rates may change as training proceeds,
- and exploration can be re-intensified at a tier when novelty, uncertainty, or policy failure reappears.

This may be better than a hard notion of "tier closure," and I want to know whether the HRL literature already supports such a view.

What I want from the literature review

Please investigate the HRL literature and report back on existing best practices, design patterns, or canonical methods relevant to the following:

1. How HRL systems decide when to keep exploring at a given level versus when to exploit/train more heavily at that level.
2. Whether mature HRL systems use hard stage transitions, soft exploration schedules, confidence thresholds, visitation thresholds, regret-style triggers, uncertainty measures, or other mechanisms.
3. Whether existing HRL methods support reopening exploration at a level that had previously become exploitation-dominant.
4. Whether these mechanisms are framed globally, per-option, per-subtask, per-level, or per-region.
5. Which parts of this are considered established best practice versus still open research problems.
6. Which existing methods are most portable to a hierarchy whose tiers are *induced abstractions* rather than naturally meaningful subtasks.

What I do *not* want

Please do *not* answer by designing a brand-new bespoke control law unless the literature genuinely has no good guidance.

Please do *not* focus primarily on my novel hierarchy-induction mechanism.

That mechanism is not the target of this request.

The target of this request is the training-control question:

> Given that some hierarchy exists, what does the HRL literature already know about balancing exploration and exploitation across hierarchical levels?

Why this matters

I would like to be able to say something mathematically and scientifically clean, such as:

> The novelty of `state_collapser` is the induction of useful non-canonical hierarchical structure in RL problems where no obvious hierarchy is available in advance. Once that structure has been induced, the package can use standard or best-practice HRL exploration/exploitation control logic.

That is a much stronger and more credible claim than pretending every aspect of the hierarchical training algorithm is new.

Desired output format

Please return:

1. a concise conceptual summary of the best available answer,
2. a categorized list of relevant HRL methods or papers,
3. an assessment of which mechanisms seem most portable to non-canonical quotient-induced hierarchies,
4. and, if possible, a final recommended control framing in one sentence, e.g. whether I should think in terms of
   - hard tier closure,
   - soft tier maturity,
   - per-tier exploration/exploitation ratios, ı-5.5 Prosome hybrid.

Where possible, prioritize authoritative or canonical sources over speculative recent variants.

---

1 ChatGPT-5.5 Pro Concise Conceptual Answer

The hierarchical reinforcement learning literature broadly supports *soft maturity*, not hard closure, as the more literature-aligned framing.

Canonical HRL methods usually treat each hierarchical level as a learner operating over its own action space: primitive actions, options, subtasks, abstract goals, or subgoal-conditioned policies. Exploration is normally handled by the learning rule at that level—epsilon-greedy, Boltzmann or

softmax exploration, entropy regularization, noise processes, intrinsic rewards, optimism, count or uncertainty bonuses, success-rate criteria, replay, or off-policy correction—rather than by declaring a tier permanently "closed."

The options framework is especially important here: options can be used with primitive actions in planning and learning, form a semi-Markov decision process, support intra-option learning, and can even be interrupted during execution when better behavior is identified; see Sutton, Precup, and Singh, "Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning".

Thus, the best scientific claim is not that HRL has a standard "tier closure" mechanism. It is closer to the following:

> Given a hierarchy, standard HRL practice is to maintain trainable controllers or value estimates at multiple temporal or abstract levels, with exploration pressure scheduled or triggered per level, per option, per subtask, per goal, or per abstract region; hard closure is mainly a curriculum or pretraining convenience, not a canonical requirement.

2 Categorized Literature-Facing Map

Category	**Representative Methods**	**What They Say About Exploration/Exploitation Control**	**Relevance to `state_collapser`**
Options / SMDP temporal abstraction	Options framework	Options are temporally extended closed-loop actions. Primitive actions and options can be combined in learning methods such as Q-learning. Intra-option learning lets an agent learn from fragments of option execution, and options can be interrupted. See Sutton, Precup, and Singh on options.	Very portable. An induced tier can be treated as an abstract option-selection or SMDP layer, provided abstract decisions lift to executable lower-level behavior.

Category	**Representative Methods**	**What They Say About Exploration/Exploitation Control**	**Relevance to `state_collapser`**
MAXQ / subtask decompositions	MAXQ	MAXQ decomposes an MDP into smaller MDPs and a hierarchical value function. It explicitly exploits state abstraction and gives conditions under which abstraction is safe. It assumes useful subgoals or subtasks are identified, which is stronger than the `state_collapser` setting. See Dietterich, "Hierarchical Reinforcement Learning with the MAXQ Value Function Decomposition".	Partly portable. The abstraction and safety lessons are useful; the assumption of programmer-defined meaningful subtasks is less portable.
End-to-end option learning	Option-Critic	Learns intra-option policies, termination functions, and the policy over options together, without supplied subgoals or extra rewards. See Bacon, Harb, and Precup, "The Option-Critic Architecture".	Highly relevant because it treats option availability, termination, and reuse as soft learned quantities rather than hard stages.

Category	**Representative Methods**	**What They Say About Exploration/Exploitation Control**	**Relevance to `state_collapser`**
Goal-conditioned HRL	h-DQN, FeUdal Networks, HIRO, HAC	h-DQN uses a top-level policy over intrinsic goals and a lower-level policy over primitive actions, making exploration occur in goal space; see Kulkarni et al., "Hierarchical Deep Reinforcement Learning: Integrating Temporal Abstraction and Intrinsic Motivation". HIRO trains both higher and lower levels off-policy and introduces correction because lower-level changes alter the effective high-level action space; see Nachum et al., "Data-Efficient Hierarchical Reinforcement Learning". HAC highlights that parallel multi-level learning is unstable because lower-level policy changes shift higher-level transition and reward functions, and it addresses this by training levels as if lower levels were already optimal; see Levy et al., "Learning Multi-Level Hierarchies with Hindsight".	Highly portable if quotient states can be used as abstract goals, regions, or latent targets. Especially useful for non-canonical hierarchies.
Optimism / confidence-based HRL	R-MAXQ, model-based HRL, MBIE-style exploration	R-MAXQ combines MAXQ abstraction with R-MAX-style efficient model-based exploration and finite-time or sample-complexity reasoning. See Jong and Stone, "Hierarchical Reinforcement Learning and R-MAX".	Very portable for discrete or countable quotient tiers, because "maturity" can be tied to abstract state-action or state-option confidence rather than semantic subtask completion.

Category	Representative Methods	What They Say About Exploration/Exploitation Control	Relevance to `state_collapser`
Skill discovery / intrinsic motivation	h-DQN, DIAYN, skill chaining, option discovery	These methods often use novelty, diversity, or intrinsic goals to create reusable behaviors. h-DQN explicitly frames sparse-reward failure as insufficient exploration and uses intrinsic goals to make exploration more efficient; see Kulkarni et al. on h-DQN.	Useful as initialization or auxiliary exploration, but less directly canonical for the tier-control question unless quotient states can serve as intrinsic goals.
State abstraction / quotient theory	MAXQ state abstraction, SMDP homomorphisms, MDP abstraction	MAXQ gives safe state-abstraction conditions. SMDP homomorphism work treats abstraction as eliminating state distinctions while preserving dynamics and explicitly extends this to hierarchical/options settings. See Ravindran and Barto, "SMDP Homomorphisms: An Algebraic Approach to Abstraction in Semi-Markov Decision Processes".	Directly relevant to the non-canonical quotient-induced hierarchy framing. It supports saying that the hierarchy-induction problem and the tier-control problem can be separated.
Modern survey consensus	HRL surveys	Surveys describe HRL as using temporal abstraction, state abstraction, options, and goal-conditioned approaches. They also emphasize that automatic end-to-end abstraction discovery remains difficult and that current methods can collapse into trivial options or whole-task options. See Pateria et al., "Hierarchical Reinforcement Learning: A Comprehensive Survey".	Supports a cautious claim: control logic is reusable, but robust automatic hierarchy discovery remains research-active, which is precisely where the novelty of `state_collapser` sits.

3 What Counts as Established Practice?

3.1 Established

HRL commonly decomposes control into multiple learners or value functions operating at different temporal or abstract scales. The options framework, MAXQ, and goal-conditioned HRL all support

this at the modeling level. Options and MAXQ are canonical; goal-conditioned HRL is a dominant modern pattern. See the options framework and MAXQ.

3.2 Established Implementation Pattern

Exploration is usually controlled by the underlying RL learner at each level: epsilon-greedy or Boltzmann exploration for value-based methods, entropy or noise for actor-critic methods, intrinsic rewards for subgoal or skill discovery, visitation thresholds or optimism for model-based methods, and replay or off-policy learning for sample reuse.

This is not usually described as "tier closure." It is described as learning a policy over options, learning subtask policies, learning goal-conditioned controllers, or maintaining confidence over abstract models.

3.3 Mature but Problem-Dependent

Confidence thresholds, visitation thresholds, and optimism are mathematically clean when the abstract layer is discrete or countable. R-MAXQ is the clearest HRL example: it inherits R-MAX-style exploration while exploiting MAXQ abstraction; see Jong and Stone on R-MAXQ.

3.4 Modern but Still Research-Active

Automatically discovering useful options, preventing option collapse, stabilizing end-to-end multi-level learning, and deciding how much exploration belongs at each level remain active research problems. Surveys note that end-to-end HRL can degrade into single-action sub-behaviors or whole-task behaviors, and that automatic option discovery can be sample inefficient; see Pateria et al.'s HRL survey.

4 Should Exploration Be Reopenable?

Yes. Reopenable exploration is more aligned with HRL practice than permanent closure.

The literature does not usually call this "reopening a tier," but the mechanisms support it. A nonzero behavior-policy exploration rate, entropy term, or action noise means the tier never fully closes. An uncertainty or count-based bonus naturally re-intensifies exploration when the agent enters poorly modeled abstract regions. A success-rate or TD-error trigger can increase sampling of a failing subgoal, option, or abstract transition.

Off-policy HRL methods such as HIRO explicitly recognize that lower-level policy drift changes the high-level learning problem, which is exactly the kind of nonstationarity that argues against irreversible closure; see Nachum et al. on HIRO.

The options framework also supports a soft view of commitment: options can be interrupted, and intra-option learning allows learning from partial executions rather than only from completed option episodes; see Sutton, Precup, and Singh on options.

5 Most Portable Mechanisms for Quotient- or Contraction-Induced Tiers

The most portable mechanisms are the ones that do *not* require human-semantic subtasks.

5.1 Most Portable

1. **Per-tier behavior-policy schedules.**

   Give each tier its own epsilon, entropy coefficient, softmax temperature, or exploration noise. This is simple, standard, and compatible with induced abstractions.

2. **Per-abstract-state or per-abstract-transition uncertainty.**

   Maintain counts, confidence intervals, posterior uncertainty, model error, TD error, or ensemble disagreement at the quotient level. This gives a clean notion of "soft maturity."

3. **Option/SMDP framing.**

   Treat abstract decisions as options or abstract actions whose execution is delegated to lower levels. This matches the options framework and avoids requiring the abstract state itself to be directly executable; see Sutton, Precup, and Singh on temporal abstraction.

4. **Goal-conditioned lower-level controllers.**

   If quotient states or clusters can be represented as goals, use a goal-conditioned value function or policy. UVFA-style methods generalize value functions over both states and goals, which fits induced abstract targets; see Schaul et al., "Universal Value Function Approximators".

5. **Off-policy replay and relabeling.**

   Use experience gathered under one abstract intent to train related intents, especially when the quotient structure supports relabeling. HIRO and HAC-style methods are relevant here because they address multi-level training instability; see HIRO and HAC.

5.2 Moderately Portable

MAXQ-style abstractions are useful if the quotient tiers can be mapped to subtasks with local termination or value decomposition. But MAXQ's classical form assumes useful subgoals are supplied, whereas `state_collapser` induces structure in problems without obvious intrinsic subtasks. See Dietterich on MAXQ.

5.3 Less Portable as Primary Control Logic

Hard pretraining phases, handcrafted subtask hierarchies, and skill-chain methods based on semantically meaningful regions are less ideal as the core claim for `state_collapser`, because they blur the novelty with human-designed hierarchy assumptions.

6 Recommended Final Framing

Use *soft tier maturity with per-tier/per-option exploration–exploitation control*, not hard tier closure.

A clean one-sentence formulation is:

> `state_collapser` induces a non-canonical hierarchical abstraction; once induced, each tier is treated as an HRL control level with a soft, adaptive exploration–exploitation schedule—driven by policy uncertainty, visitation, prediction error, success/failure, or off-policy replay—rather than being permanently closed after a fixed exploration phase.

This lets the novelty be the hierarchy induction, while the training-control layer uses recognizable HRL design patterns.